JAVA LOOP & ARRAY DATA PROCESSOR

OBJECTIVE

A comprehensive data processor that iterates through patient health records and investment portfolios using loops and arrays to make automated decisions. This project is a critical evolution from Set 1, moving from individual variables to handling collections of data. It simulates real-world AI healthcare monitoring systems and financial modeling tools that process batch data to provide real-time alerts and insights. The primary goal is to use efficient Java iteration (for-each and standard for-loops) to automate the analysis of multiple data points simultaneously, which is essential for high-paying AI engineering roles.

STEPS

1. **Review and Planning:** Reviewed all Java source code from Day 1 to Day 9 to ensure a solid understanding of loops, arrays, and conditional logic.
2. **Project Structure:** Integrated all previous foundational concepts into a single, cohesive "Java Loop & Array Data Processor" within the *dilip-java-dsa-ai-portfolio* project.
3. **Data Initialization:** Initialized an integer array `patientHR` for heart rates and a double array `investmentReturns` to store portfolio performance, ensuring precise data type selection for accuracy.
4. **Loop Implementation:** Implemented `for` loops and `for-each` loops to traverse the data collections efficiently.
5. **Healthcare Logic:** Applied conditional logic within the patient loop to flag heart rates above 80 with a "High - AI Healthcare Alert."
6. **Financial Logic:** Applied conditional logic within the investment loop to identify returns over 10% as "Profitable - AI Modeling Alert."
7. **Trial and Error:** Performed manual testing by changing array values 4-5 times and fixing deliberate mistakes to build persistence and logical reasoning at an average learning pace.

EXPECTED OUTPUT

- The console prints the specific heart rate for each patient along with their risk status (e.g., "Patient 1 HR: 72 - Normal").
- The AI healthcare alert triggers automatically for any patient with a heart rate exceeding the defined threshold.
- The investment return processing outputs the specific percentage and flags high-yield investments as "Profitable."
- A clean, structured console output that demonstrates the system's ability to handle multiple data entries using Java collections.